Rijndael

Write a Python program that implements a Rijndael cipher that takes a ciphertext as input from **stdin** and outputs the correct plaintext and corresponding key used in the cipher to **stdout**. You will be provided with a dictionary of words that is **guaranteed** to contain the key used to generate the ciphertext. Note that Rijndael is the cipher that was selected for the Advanced Encryption Standard (AES).

Since you will be playing the part of cryptanalysts, your program will need to generate candidate plaintexts using keys in the provided dictionary. Once candidate plaintexts are generated, you must **efficiently** filter out invalid ones using a method of your choice. As in previous programs, one way is to compare “words" in candidate plaintexts with words in the provided dictionary. Note that you may want to “normalize" words in candidate plaintexts and the dictionary (e.g., remove punctuation, convert to lowercase, etc). If enough words match (say, three-fourths of them), a candidate plaintext becomes likely. You can also consider using the average length of words in English US to help identify candidate plaintexts.

In general, you will need to implement the following **algorithm**:

```
1: C ← read ciphertext from stdin as bytes
2: D ← read the dictionary file as a list of possible keys
3: for all candidate_key in D do
4:     K ← convert candidate_key to bytes if needed
5:     K ← SHA-256(K)
6:     IV ← first 16 bytes of C
7:     E ← all bytes of C after the first 16 bytes
8:     M ← AES-CBC-Decrypt(E, K, IV)
9:     remove padding characters from the end of M if needed
10:    if the expected output is readable text then
11:        convert M from bytes to a string
12:        remove punctuation and split M into words
13:        count how many words in M also appear in the dictionary
14:        if count / total_words > THRESHOLD then
15:            print candidate_key
16:            print M
17:            stop
18:    else if the expected output is a PDF or binary file then
19:        if M contains the PDF header "%PDF" then
20:            print candidate_key to stderr
21:            write M as raw bytes to stdout
22:            stop
```

Notes and Requirements:

- Submit your source code only. I will provide my own ciphertext to test with;
- Read the ciphertext from stdin;
- Write the plaintext and key to stdout;
- Comment your source code appropriately; and
- The ciphertext could contain multiple lines.

Please, no GUIs. Make this a command line application without frills that I can execute at the command line as illustrated below via several sample runs of my program:

```
Mtahat> py rijndael.py < ciphertext-1
Key = heartburn
The lady said, "Oh my! You have nice eyes. Are they yours?"
I laughed.

Mtahat> py rijndael.py < ciphertext-2
Key = Mercury
Never attribute to malice that which is adequately ex-
plained by stupidity.
(Hanlon's razor)
```